

**UNIVERSITY OF SCIENCE, VNU - HCM
FACULTY OF INFORMATION TECHNOLOGY**



INDIVIDUAL ASSIGNMENT REPORT – IA02

TOPIC: WEB-BASED CALCULATOR

SUBJECT: WEB DEVELOPMENT

TEACHER: NGUYỄN HUY KHÁNH

Students:

NGUYỄN ĐOÀN XUÂN THU

MSSV: 23120170

October 22, 2025

MỤC LỤC

I.	Functional Specifications.....	3
1.	Purpose and Scope	3
2.	Supported Features (Equivalent to Windows 11 Basic Mode).....	3
3.	Handling of Inputs, Operators, and Display	4
4.	Assumptions.....	4
II.	Non-Functional Specifications.....	5
III.	Acceptance Criteria.....	6
IV.	Testing Plan	6
V.	Prompt Engineering (AI assistance)	8
VI.	Work achievement	9

I. Functional Specifications

1. Purpose and Scope

The purpose of this project is to build a web-based calculator that replicates the *Basic Mode* of the Windows 11 Calculator using *React*.

The application performs standard arithmetic operations and provides a *responsive, accessible, and user-friendly* interface that runs entirely in the browser — with no backend or external APIs required.

Scope:

- Single-page web application (SPA) using React.
- Fully client-side logic and computation.
- Mimics the look, feel, and core functionality of Windows 11 Basic Mode.
- Publicly hosted on a deployment platform: Vercel with link: <https://calculator-seven-chi-67.vercel.app/>
- Source code: <https://github.com/thu2005/calculator.git>

2. Supported Features (Equivalent to Windows 11 Basic Mode)

Feature	Description
Number Input	Supports digits 0-9 and a decimal point (.).
Basic Arithmetic	Addition (+), subtraction (-), multiplication (×), and division (÷).
Percentage (%)	Calculates percentage values relative to the current or previous operand.
Square Root ($\sqrt{}$)	Computes the square root of the current input.
Square (x^2)	Computes the square of the current input.
Reciprocal ($1/x$)	Computes the reciprocal (1 divided by the current input).
Negate ($\pm$)	Toggles between positive and negative for the current number.
Clear Entry (CE)	Clears only the current input value.
Clear (C)	Clears all entries and resets the calculator.
Backspace ($\leftarrow$)	Deletes the last character of the current input.
Memory Store (MS)	Stores the current input value into calculator memory.
Memory Recall (MR)	Retrieves the most recently stored value from the memory.
Memory Add (M+)	Adds the current input value to the first memory slot.
Memory Subtract (M-)	Subtracts the current input value from the first memory slot.
Memory Clear (MC)	Clears all stored memory values.
History Panel	Automatically records every performed calculation (not only those confirmed with “=”).
Memory Panel	Displays the current memory stack for easier tracking and management.
Display Panel	Shows both the current value and (if applicable) the previous expression.
Keyboard Support	Allows entering numbers and operators from the keyboard.
Responsive UI	Works on both desktop and mobile screens.

3. Handling of Inputs, Operators, and Display

- Numeric Input:
 - When a user presses a digit or a decimal point, the current input string updates accordingly. Leading zeros are handled automatically, and multiple decimal points are prevented.
- Operator Input:
 - If no prior value exists, the current input becomes the stored previous value, and the selected operator is saved.
 - If both a previous value and a current input exist, the calculator computes the intermediate result and updates the display.
 - Pressing = executes the operation and displays the final result.
- Display behavior:
 - The top line shows the previous expression (e.g., $12 +$).
 - The main line shows the current input or the result after pressing =.
 - When an error occurs (e.g., division by zero or invalid square root), the display shows “Error” while remaining functional.
 - CE, C and Backspace:
 - CE clears only the current input.
 - C resets the calculator entirely.
 - Backspace removes the last digit or character of the current input.
- Special Operations:
 - $\sqrt{}$: Applies to the current input; returns “Error” if the input is negative.
 - x^2 : Squares the current input immediately and updates the display.
 - $1/x$: Reciprocates the current input immediately and updates the display; returns “Error” if the input is zero.
 - $\pm$: Multiplies the current input by -1 and updates the display immediately.
 - %: Behaves as “percentage of the previous value” (e.g., $50 + 10 \% = 55$).
- Memory Handling:
 - MS: stores the current input into memory.
 - MR: recalls the most recently stored memory value.
 - M+/M-: add or subtract the current input from the first memory slot.
 - MC: clears all stored memory values.
 - All memory values are displayed in the Memory panel as a stack for easy tracking.
- History Recording:
 - Each performed operation (not only those confirmed by “=”) is saved in the History panel, which allows users to review past calculations.

4. Assumptions

- Operator Order:
 - The calculator evaluates expressions strictly from left to right, without applying standard arithmetic precedence.

- Each operation is executed as soon as the next operator is entered, consistent with the behavior of Windows 11 Calculator in Basic Mode.
- Rounding & Precision:
 - JavaScript’s floating-point arithmetic (IEEE 754) is used. Displayed results are limited to 12 significant digits.
- Error Handling:
 - Division by zero, reciprocal of zero ($1/0$), and invalid square roots ($\sqrt{\text{of negative numbers}}$) return “Error” instead of crashing the application.
 - Invalid number formats (e.g., multiple decimal points) are automatically prevented. The calculator remains functional after displaying an error message.
- Keyboard Input:
 - If implemented, numeric keys (0–9), operator keys, Enter (=), Backspace, and Escape/Delete (for C/CE) are supported.
- Memory & History Behavior:
 - Memory operations (MS, MR, M+, M–, MC) are stored and updated sequentially, maintaining a visible memory stack.
 - The History panel records all executed operations in real time and is cleared only when the calculator is reset or the clear button is pressed.

II. Non-Functional Specifications

- Performance:
 - The core logic is implemented within a lightweight React hook performing synchronous calculations, ensuring instant updates on button press. History and memory updates operate in $O(1)$ time, with no unnecessary re-renders or expensive effects detected.
- Usability:
 - The interface closely follows the Windows Calculator layout, providing clear grouping of operators and memory functions. It displays live expressions and calculation history in real time.
- Cross-Browser Compatibility:
 - The application uses standard React, DOM APIs, and basic CSS without browser-specific dependencies. It behaves consistently across Chrome, Edge, Firefox, and Safari.
- Responsiveness:
 - The grid layout adapts automatically, collapsing to a single centered calculator below 1024 px and optimizing keypad height for mobile devices. Both desktop and handheld experiences are fully supported.
- Reliability & Maintainability:
 - Core functionality resides in the useCalculator hook, separating logic from UI. State transitions are centralized and easily testable, with helper utilities encapsulating arithmetic operations. This modular structure supports straightforward extension and long-term maintainability.

III. Acceptance Criteria

To verify that the Web-Based Calculator meets all specified requirements, the following criteria must be satisfied:

- Arithmetic Accuracy:
 - All basic and extended arithmetic operations (including $\sqrt{}$, x^2 , $1/x$, $\%$, and memory functions) return correct results consistent with the Windows 11 Calculator.
- Operator Execution Order:
 - Expressions are evaluated strictly from left to right, with each operation executed immediately after the next operator is entered.
- Input and Control Behavior:
 - The CE, C, and Backspace buttons perform as expected: CE clears only the current entry, C resets the entire calculator, and Backspace removes the last input character.
- Display Updates:
 - Both the current value and the previous expression are displayed accurately after each input or operation, including error messages when invalid operations occur.
- Interface Consistency:
 - The calculator layout and design remain stable and visually consistent across supported browsers and devices.
- Deployment Validation:
 - The deployed version is publicly accessible, fully functional, and responsive under real-world usage scenarios.

IV. Testing Plan

Testing Method: Automated testing implemented using React Testing Library. Each test simulates user interactions and verifies calculator logic, display updates, and state transitions.

- Basic Number Input & Display (7 test cases)
 - Initialize with zero: Verify calculator initializes with value "0"
 - Multiple digit input: Verify multiple consecutive digits input (123)
 - Leading zero handling: Verify handling of leading zero ($0 + 5 = 5$)
 - Decimal point input: Verify decimal point input (1.5)
 - Prevent multiple decimal points: Verify prevention of multiple decimals ($1.5.8 \rightarrow 1.58$)
 - Decimal with leading zero: Verify handling of leading zero before decimal ($.5 \rightarrow 0.5$)
- Basic Operations (8 test cases)
 - Addition: $2 + 3 = 5$
 - Subtraction: $7 - 4 = 3$
 - Multiplication: $6 * 8 = 48$
 - Division: $15 / 3 = 5$
 - Chain operations: $2 + 3 * 4 = 20$ (evaluated sequentially from left to right)
 - Decimal operations: $1.5 + 2.3 = 3.8$

- Division by zero: $5 / 0 = \text{Error}$
- Advanced Operations (7 test cases)
 - Square: $5^2 = 25$
 - Reciprocal: $1/4 = 0.25$
 - Square root: $\sqrt{16} = 4$
 - Reciprocal of zero: $1/0 = \text{Error}$
 - Square root of negative: $\sqrt{(-4)} = \text{Error}$
 - Decimal square: $(2.5)^2 = 6.25$
 - Decimal square root: $\sqrt{6.25} = 2.5$
- Memory Operations (8 test cases)
 - Memory Store (MS): Store value 123 into memory
 - Memory Add (M+): Add $50 + 25 = 75$ in memory
 - Memory Subtract (M-): Subtract $100 - 30 = 70$ in memory
 - Memory Recall (MR): Retrieve stored value to display
 - Memory Clear (MC): Clear all memory slots
 - Decimal memory operations: Handle decimals correctly in memory ($3.14 + 2.86 = 6$)
 - Empty memory recall: Handle recall from empty memory safely
 - Complex memory sequence: Test operation chain (MS $\rightarrow$ M+ $\rightarrow$ M- $\rightarrow$ MR)
- History Management (3 test cases)
 - Add to history: Record $5 + 3 = 8$ in history
 - Multiple calculations: Store multiple results in correct order (stack behavior)
 - Clear history: Remove all entries when cleared

Test Statistics

- Total Test Cases: 33
- Test Categories: 5
- Coverage Areas:
 - Number input validation
 - Basic arithmetic operations
 - Advanced mathematical functions
 - Memory management
 - History tracking
 - Error handling
 - Decimal number support
 - Edge case validation

Key Testing Patterns Used

- Individual act() calls: Each UI interaction wrapped in a separate act()
- State validation: Validate currentInput and displayExpression after each action
- Error handling: Verify cases like divide by zero, negative square roots, reciprocal of zero
- Edge cases: Handle empty memory, multiple decimals, leading zeros

- Complex workflows: Cover chain operations, memory sequences, and history tracking

```
PS D:\Web\calculator> npm test
✓ src/__tests__/useCalculator.test.js (31 tests) 76ms
✓ Calculator Tests > 1. Basic Number Input & Display > should initialize with zero 20ms
✓ Calculator Tests > 1. Basic Number Input & Display > should handle multiple digit input 3ms
✓ Calculator Tests > 1. Basic Number Input & Display > should handle leading zero 2ms
✓ Calculator Tests > 1. Basic Number Input & Display > should handle decimal point 2ms
✓ Calculator Tests > 1. Basic Number Input & Display > should prevent multiple decimal points 2ms
✓ Calculator Tests > 1. Basic Number Input & Display > should handle decimal point with leading zero 2ms
✓ Calculator Tests > 2. Basic Operations > should perform addition: 2 + 3 = 5 3ms
✓ Calculator Tests > 2. Basic Operations > should perform subtraction: 7 - 4 = 3 2ms
✓ Calculator Tests > 2. Basic Operations > should perform multiplication: 6 * 8 = 48 2ms
✓ Calculator Tests > 2. Basic Operations > should perform division: 15 / 3 = 5 2ms
✓ Calculator Tests > 2. Basic Operations > should handle chain operations: 2 + 3 * 4 = 14 2ms
✓ Calculator Tests > 2. Basic Operations > should handle decimal operations: 1.5 + 2.3 = 3.8 1ms
✓ Calculator Tests > 2. Basic Operations > should handle division by zero: 5 / 0 = Error 1ms
✓ Calculator Tests > 3. Advanced Operations > should calculate square: 5² = 25 1ms
✓ Calculator Tests > 3. Advanced Operations > should calculate reciprocal: 1/4 = 0.25 1ms
✓ Calculator Tests > 3. Advanced Operations > should calculate square root: √16 = 4 1ms
✓ Calculator Tests > 3. Advanced Operations > should handle reciprocal of zero: 1/0 = Error 2ms
✓ Calculator Tests > 3. Advanced Operations > should handle square root of negative number: √(-4) = Error 2ms
✓ Calculator Tests > 3. Advanced Operations > should handle decimal square: (2.5)² = 6.25 2ms
✓ Calculator Tests > 3. Advanced Operations > should handle decimal square root: √6.25 = 2.5 1ms
✓ Calculator Tests > 4. Memory Operations > should store value in memory: MS 2ms
✓ Calculator Tests > 4. Memory Operations > should add value to memory: M+ 2ms
✓ Calculator Tests > 4. Memory Operations > should subtract value from memory: M- 2ms
✓ Calculator Tests > 4. Memory Operations > should recall memory value: MR 2ms
✓ Calculator Tests > 4. Memory Operations > should clear memory: MC 2ms
✓ Calculator Tests > 4. Memory Operations > should handle memory operations with decimal numbers 2ms
✓ Calculator Tests > 4. Memory Operations > should handle memory recall when memory is empty 1ms
✓ Calculator Tests > 4. Memory Operations > should handle complex memory operations sequence 2ms
✓ Calculator Tests > 5. History Management > should add calculation to history 2ms
✓ Calculator Tests > 5. History Management > should accumulate multiple calculations in history 2ms
✓ Calculator Tests > 5. History Management > should clear history when requested 2ms

Test Files 1 passed (1)
Tests 31 passed (31)
```

V. Prompt Engineering (AI assistance)

- AI Prompts Used:
 - ChatGPT was used for both code and documentation generation. Prompts focused on improving calculator logic (e.g., handling operations, memory stack behavior, and error handling) and structuring report sections such as *Functional Specifications* and *Testing Plan*. GitHub Copilot was also used to generate test cases and suggest small code fragments.
- AI Assistance in Development:
 - AI tools supported the development process by providing ideas for code structure, debugging calculation issues, and optimizing React state management. They also helped refine documentation language and maintain consistency throughout the report.
- Reflection and Learning:
 - Through this process, I learned how to design precise prompts, interpret AI-generated outputs critically, and integrate them effectively with my own reasoning. AI served as a collaborative assistant, not a replacement for manual coding or verification.
- Verification:
 - All AI-generated code, logic, and documentation were reviewed, tested, and verified by me before inclusion in the final project.

VI. Work achievement

Criteria	Description	Weight
Functional Specifications	Completeness and accuracy of described calculator features	15%
Non-Functional Specifications	Clarity and coverage of usability, performance, and responsiveness	10%
Acceptance Criteria	Well-defined, testable, aligned with requirements	10%
Testing Plan	Clear test cases, expected results, organized structure	10%
Source Code Quality	Correctness, readability, structure, and best practices	25%
Public Hosting & Deployment	Calculator successfully deployed, accessible online, and functions as expected	10%
Prompt Engineering (AI Assistance)	Clear inclusion of prompts, responsible AI use, reflection quality	10%
Documentation & Presentation	Overall clarity, structure, and professionalism of the report	10%

Total: 100%.